ARCHITEKTURA I MODEL DANYCH V1.0

DRAFT

ClientOps

Platforma CRM nowej generacji

Diagram logiczny, modele danych i UI

Szczegółowa architektura systemu: warstwy danych, relacje między bytami, flow sprzedaży, Kanban leadów, podział tickets/sla_incidents oraz struktura UI panelu admina i klienta.

DATA	WERSJA	DOKUMENT	POWIĄZANY Z
Marzec 2026	1.0.0	Architektura	Spec v1.0.0

Spis treści

Dokument zawiera 7 sekcji

1 Podział systemu na warstwy

2 Diagram relacji między bytami

3 Flow sprzedaży

4 Kanban leadów — kolumny i statusy

5 Tickets vs SLA Incidents

6 Audyty i analizy SEO — relacja

7 UI — struktura menu

1

ARCHITECTURE

Podział systemu na warstwy

System podzielony na trzy warstwy logiczne. Każda warstwa obsługuje inny obszar biznesowy i zależy od warstwy poprzedniej — nie można mieć projektu bez klienta, nie można mieć klienta bez kontaktu.

WARSTWA 1 — TOŻSAMOŚĆ

`users` `clients` `contacts`

WARSTWA 2 — SPRZEDAŻ

`leads` `lead_activities` `offers`

WARSTWA 3 — REALIZACJA

`projects` `documents` `seo_analyses` `audits` `notifications` `tickets` `sla_policies` `sla_incidents`

Minimalny zestaw tabel na start

Kolejność wdrożenia odpowiada warstwom — nie można tworzyć tabeli wyższej warstwy bez fundament niższej.

TABELA	WARSTWA	OPIS
`users`	1	Użytkownicy systemu — admini, managerowie, pracownicy, klienci
`clients`	1	Firmy / organizacje — główny byt systemu CRM
`contacts`	1	Osoby kontaktowe w firmach klientów (wiele per klient)
`leads`	2	Szanse sprzedażowe powiązane z firmą lub kontaktem
`lead_activities`	2	Log aktywności na leadzie: telefon, email, spotkanie, notatka
`offers`	2	Oferty handlowe — krok między leadem a klientem
`projects`	3	Realizacje dla klientów — agregują wszystkie zasoby operacyjne
`documents`	3	Dokumenty powiązane z projektami / klientami
`seo_analyses`	3	Analizy SEO — samodzielne lub jako składowa audytu
`audits`	3	Audyty (SEO, techniczne, UX) — byt nadrzędny grupujący analizy
`notifications`	3	Powiadomienia systemowe dla użytkowników

TABELA	WARSTWA	OPIS
`tickets`	3	Zgłoszenia tworzone przez klientów (sfera ludzka)
`sla_policies`	3	Polityki SLA — czasy odpowiedzi i rozwiązania per priorytet
`sla_incidents`	3	Naruszenia SLA wykryte automatycznie przez system

Kluczowa zasada architektury

Nie łącz wszystkiego w jeden byt. Lead, klient i projekt to trzy oddzielne stany relacji biznesowej. Lead staje się klientem przez konwersję — projekt jest tworzony dla klienta, nigdy dla leadu.

2

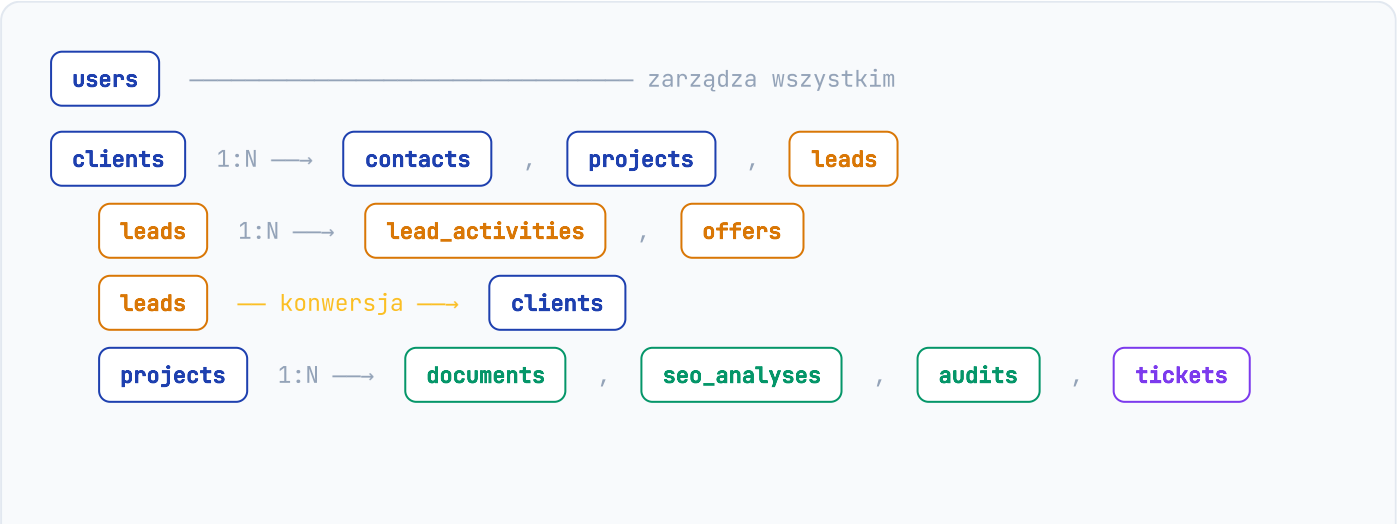
DATA MODEL

Diagram relacji między bytami

Relacje główne

RELACJA	TYP	OPIS
clients → contacts	1:N	Jedna firma ma wiele osób kontaktowych
clients → projects	1:N	Jeden klient ma wiele projektów
clients → leads	1:N	Firma może mieć wiele szans sprzedażowych
contacts → leads	1:N	Konkretna osoba może być powiązana z leadem
leads → lead_activities	1:N	Historia aktywności na leadzie
leads → offers	1:N	Lead może mieć wiele wersji oferty
leads → clients	konwersja	Wygrany lead przekształca się w klienta
projects → documents	1:N	Projekt agreguje dokumenty
projects → seo_analyses	1:N	Projekt agreguje analizy SEO
projects → audits	1:N	Projekt agreguje audyty
audits → seo_analyses	1:N (opcja)	Audyt może grupować wiele analiz SEO
projects → tickets	1:N	Zgłoszenia dotyczą konkretnych projektów
tickets → sla_incidents	1:N	Ticket może wygenerować wiele naruszeń SLA
projects → sla_incidents	1:N	Naruszenie deadline projektu też tworzy incydent

Schemat wizualny



tickets — system ⟶ sla_incidents

3

PROCESS

Flow sprzedaży

Kompletny proces od pierwszego kontaktu do aktywnej realizacji projektu. Każdy krok to osobny byt w systemie — nie skróty, nie aliasy.

Główny flow

Kontakt / firma → Lead (zimny) → Lead (ciepły) → Oferta → Klient → Projekt

Projekt → Dokumenty + Analizy SEO + Audyty + SLA + Powiadomienia

Krok "Oferta" — nowy element względem spec v1.0.0

Oferta to osobny byt między leadem a klientem. Lead może mieć wiele wersji oferty (renegocjacje). Akceptacja oferty wyzwala konwersję leadu na klienta i opcjonalne automatyczne utworzenie projektu.

Schematy tabel nowych bytów

```
-- Kontakty (osoby w firmach)
contacts
  - client_id
  - first_name, last_name
  - email, phone
  - position          -- stanowisko
  - is_primary: bool  -- główny kontakt
  - notes
```

```
-- Aktywności na leadzie
lead_activities
  - lead_id
  - user_id           -- kto wykonał
  - type: call | email
      | meeting | note
      | status_change
  - description
  - occurred_at
```

```
-- Oferty handlowe
offers
  - lead_id
  - client_id (nullable)   -- wypełniane przy konwersji
  - title
  - status: draft | sent | accepted | rejected | expired
  - valid_until
  - document_id (nullable) -- powiązanie z edytorem dokumentów (TipTap)
  - accepted_at
  - converted_to_client_at
```

Flow admina krok po kroku

KROK	AKCJA ADMINA	CO SIĘ DZIEJE W SYSTEMIE
1	Tworzy lead	Rekord w `leads`, status: new, opcjonalne powiązanie z `contacts`

KROK	AKCJA ADMINA	CO SIĘ DZIEJE W SYSTEMIE
2	Kontaktuje się	Wpis w `lead_activities` (type: call/email), zmiana statusu leadu
3	Wysyła ofertę	Rekord w `offers` (status: sent), opcjonalnie dokument TipTap jako PDF
4	Oferta zaakceptowana	`offers.status` → accepted, lead → status: wygrany
5	Konwertuje lead → klient	Tworzony `clients`, `leads.converted_at`, kopiowane kontakty
6	Tworzy projekt	Rekord w `projects` z `client_id`, przypisanie SLA policy

4

UI / PROCESS

Kanban leadów — kolumny i statusy

Kanban leadów ma 7 kolumn odzwierciedlających pełny cykl sprzedażowy. Statusy są enumeracją w tabeli `leads` — zmiana kolumny to zmiana statusu.

Nowy	Skontaktowany	Brak odpowiedzi	W rozmowie	Oferta	Wygrany	Utracony
Firma ABC	XYZ Sp. z o.o.	Alpha Sp. k.	Beta Corp	Delta S.A.	Epsilon	Zeta Inc.
Jan Kowalski			Gamma Ltd			

Statusy jako enum w bazie

```
-- leads.status
enum LeadStatus {
    new,            // Nowy
    contacted,      // Skontaktowany
    no_response,    // Brak odpowiedzi
    in_discussion,  // W rozmowie
    offer_sent,     // Oferta
    won,            // Wygrany → wyzwala konwersję do klienta
    lost            // Utracony
}
```

Ujednolicony model Kanban

Ten sam board obsługuje trzy typy kart dzięki polymorphic relations. Zmiana statusu leadu na "won" lub "lost" to jedyna kolumna terminalna — karta znika z głównego widoku.

```
kanban_cards
  - board_id, column_id, position
  - cardable_type: "Lead" | "Ticket" | "ProjectTask"
  - cardable_id
  - assigned_to, due_at

-- Zmiana statusu leadu automatycznie przesuwa kartę:
-- LeadStatusChanged event → MoveKanbanCardListener
```

Aktywności na leadzie w widoku karty

TYP AKTYWNOŚCI	IKONA	CO ZAPISUJE
`call`	📞	Data, czas trwania, notatka z rozmowy
`email`	✉️	Temat, kierunek (wysłany/odebrany), notatka
`meeting`	🤝	Data spotkania, miejsce/link, uczestnicy, notatka
`note`	📝	Dowolna notatka tekstowa

TYP AKTYWNOŚCI	IKONA	CO ZAPISUJE
`status_change`	🔄	Poprzedni i nowy status, automatycznie przez system

5

ARCHITECTURE

Tickets vs SLA Incidents

Dwa osobne byty o różnym źródle pochodzenia i celu. Powiązane — ale niezależne.

tickets — sfera ludzka

Twórca	klient (ręcznie)
Wyzwalacz	człowiek klika "zgłoś"
Cel	obsługa zgłoszenia
Widoczny dla	klient + admin
Komentarze	tak (+ wewnętrzne)
Kanban	tak → karta admina

sla_incidents — sfera systemowa

Twórca	system (automatycznie)
Wyzwalacz	upływ czasu / brak akcji
Cel	alert o naruszeniu umowy
Widoczny dla	admin / manager
Komentarze	nie
Kanban	zmiana koloru ticketu

Co wyzwala sla_incident

BREACH_TYPE	PODMIOT	WARUNEK
`response`	Ticket	`sla_response_due_at < now()` AND brak pierwszej odpowiedzi
`resolution`	Ticket	`sla_resolution_due_at < now()` AND ticket nie zamknięty
`deadline`	Project	`projects.deadline < now()` AND status != completed

Schemat tabel

```
-- Zgłoszenia klientów
tickets
  - client_id, project_id
  - created_by
  - title, description
  - priority: low|medium
              |high|critical
  - status: new|open|in_progress
            |waiting|resolved|closed
  - sla_policy_id
  - sla_response_due_at
  - sla_resolution_due_at
  - first_responded_at
  - resolved_at
  - kanban_card_id
```

```
-- Naruszenia SLA (system)
sla_incidents
  - subject_type:
      "Ticket" | "Project"
  - subject_id
  - breach_type:
      response | resolution
      | deadline
  - breached_at
  - detected_at
  - notified: bool
  - acknowledged_by
  - acknowledged_at
  - resolved_at
```

Scheduled job — monitoring co 5 minut

```
// app/Console/Commands/CheckSlaBreaches.php
Schedule::job(CheckSlaBreachesJob::class)→everyFiveMinutes();

CheckSlaBreachesJob skanuje:
├── tickets: sla_response_due_at < now() AND first_responded_at IS NULL
```

```
│     → nowy sla_incident (response) + SMS do admina (jeśli critical)
│     → ticket na Kanban → kolor czerwony
│
├── tickets: sla_resolution_due_at < now() AND resolved_at IS NULL
│     → nowy sla_incident (resolution) + email do managera
│
└── projects: deadline < now() AND status ≠ 'completed'
      → nowy sla_incident (deadline) + powiadomienie in-app
```

Jeden ticket może mieć wiele sla_incidents

Np. najpierw `response_breach` (nie odpowiedziano w czasie), potem `resolution_breach` (nie rozwiązano w czasie). Każdy incydent jest osobnym rekordem z własnym `detected_at`.

6

MODULE

Audyty i analizy SEO — relacja

Analiza SEO może być bytem samodzielnym (uruchamianym ad hoc) lub składową audytu (grupowaną razem z innymi typami analiz). Obie ścieżki są obsługiwane przez jeden model z opcjonalnym `audit_id`.

Dwa tryby użycia

Analiza SEO — samodzielna

Admin uruchamia analizę dla projektu bez tworzenia audytu. `seo_analyses.audit_id = null`. Widoczna w sekcji "Analizy SEO" projektu. Typowy przypadek: miesięczny monitoring pozycji.

Analiza SEO — składowa audytu

Audit grupuje wiele analiz i typów (SEO + techniczne + UX). `seo_analyses.audit_id = 42`. Wyniki agregowane w raporcie audytu z jednym scoringiem. Typowy przypadek: kompleksowy audyt kwartalny.

Schemat tabel

```
-- Byt nadrzędny — grupuje analizy różnych typów
audits
  - project_id
  - type: seo | technical | ux | security | full
  - status: pending | running | completed | failed
  - overall_score     -- 0-100, agregat wszystkich składowych
  - started_at, completed_at
  - created_by

-- Analiza SEO — samodzielna LUB składowa audytu
seo_analyses
  - project_id
  - audit_id (nullable)  -- null = samodzielna, int = część audytu
  - status: pending | running | completed | failed
  - score                -- 0-100
  - keywords_count
  - issues_count
  - gsc_connected: bool -- czy dane z Google Search Console
  - ga4_connected: bool -- czy dane z Google Analytics 4
  - completed_at

-- Błędy wykryte w analizie
seo_issues
  - seo_analysis_id
  - type: missing_meta | broken_link | duplicate_content | ...
  - severity: critical | warning | info
  - url
  - description
  - fixed_at (nullable)
```

Flow uruchamiania

Admin klika "Uruchom analizę" → Tworzony seo_analyses → Pipeline jobów (Horizon) → Wyniki zapisane → Powiadomienie

Dlaczego nie jeden model "seo_audits"?

Rozdzielenie `audits` od `seo_analyses` pozwala w przyszłości dodać inne typy analiz (UX, security, performance) bez zmiany struktury. Audyt staje się "folderem" — analiza SEO to jeden z dokumentów w tym folderze.

7

UI

Struktura menu — panel admina i klienta

PANEL ADMINA — APP.CLIENTOPS.PL
• Dashboard
• Leady
• Klienci
• Kontakty
• Projekty
• Oferty
• Dokumenty
• Analizy SEO
• Audyty
• Zgłoszenia (Tickets)
• SLA / Naruszenia
• Powiadomienia
• Użytkownicy
• Ustawienia

PANEL KLIENTA — {SLUG}.CLIENTOPS.PL
• Dashboard
• Moje projekty
• Dokumenty
• Analizy SEO
• Audyty
• Zgłoszenia
• SLA — status umowy
• Powiadomienia
• Profil

Różnice w widokach

SEKCJA	ADMIN WIDZI	KLIENT WIDZI
Leady	Wszystkie leady, Kanban, aktywności	— (niedostępne)
Oferty	Pełne zarządzanie ofertami	— (niedostępne)
Kontakty	Wszystkie kontakty per klient	Własne dane kontaktowe
Projekty	Wszystkich klientów, pełne dane	Tylko swoje, ograniczony widok
Zgłoszenia	Wszystkie tickets, komentarze wewnętrzne	Tylko swoje, bez wewnętrznych
SLA	Incydenty, konfiguracja polityk	Status umowy, czy SLA jest spełniane
SEO / Audyty	Pełne dane, uruchamianie analiz	Wyniki i raporty (read-only)